Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Facade (yapısal kalıp)
  - Amaç: çeşitli alt sistemlerdeki farklı arayüzleri tek, ortak ve daha üst düzey bir arayüzde toplamak. İstemci, hangi alt sistemin hangi metodunun çağrılacağını bilmeden gereken bütün çağrıları tek bir erişim noktasından yapar.
  - Facade sınıfının metotları, alt sistemdeki ilgili sınıfların ilgili metotlarını çağırıp sonucu geri döndürür. Hoca kalıbın başka bir numarası olmadığını, ayrı bir örnek gerektirmediğini söyledi.
  - Hoca, adın İtalyanca kökenli "faça" (yüz, sima) sözcüğüyle hatırlanabileceğini belirtti.
- Flyweight: amaç, örnek ve sorun (ekrandan, 00:10:00)
  - Amaç: paylaşım yoluyla çok sayıda basit nesnenin kullanımını desteklemek.
  - Örnek: font ve biçimlendirme bilgilerine göre metni çok net gösteren (excellent rendering) bir metin düzenleme programı. Harfler grafik nesneleri olarak gösterilir; metin sütun ve satırlar içindeki harfler olarak ele alınır.
  - Sorun: her harf ayrı bir nesne olursa küçük bir belgede bile binlerce nesne oluşur; bunların bellekte ve ikincil saklama ortamında kapladığı yer aşırı olur. Slayttaki örnek: bu dersin üç sayfalık öneri formunda bile boşluklarla birlikte 5000 kadar harf vardır.
  - Çözüm fikri: her satır ve sütun konumunda harf nesnesi yerine yalnız harfi simgeleyen küçük bir anahtar (bir işaretçi gibi) tutulur. Her harfin grafik nesnesinden bir tane oluşturulur ve ortak bir havuzda (flyweight pool) paylaşılır. Satır, sütun ve karakterler ortak bir glif hiyerarşisinde toplanır; hoca bunun bir önceki derste görülen Composite kalıbına benzediğini söyledi.
- Flyweight kalıp yapısı (ekrandan, 00:18:00)
  - `FlyweightFactory` (`GetFlyweight(key)`), havuzu (`flyweights`) tutar: anahtara karşılık flyweight varsa onu döndürür, yoksa yenisini oluşturur, havuza ekler ve döndürür.
  - Soyut `Flyweight` (`Operation(extrinsicState)`): bütün metotları soyut olduğu için Java'da arayüz olarak da kodlanabilir.
  - `ConcreteFlyweight` (`intrinsicState` tutar) ve paylaşılmayan `UnsharedConcreteFlyweight` (`allState` tutar). `Client` hem fabrikayı hem flyweight nesnelerini kullanır.
  - Hoca, fabrika bulunamayan anahtar için duruma göre `null` döndürebileceğini, nesneyi bir yerden yükleyebileceğini ya da yüklenene kadar bir Proxy nesnesi döndürebileceğini söyledi.
- İçsel (intrinsic) ve dışsal (extrinsic) durum bilgisi (ekrandan, 00:20:00)
  - İçsel durum: flyweight'in değişmeyen, paylaşılan kısmı; örnekte harfin grafiği. Dışsal durum: kullanıldığı yere göre değişen ve dışarıdan metot parametresiyle verilen kısım; örnekte harfin satır ve sütunu.
  - Alternatif terminoloji: intrinsic → immutable, extrinsic → mutable. İçsel kısmı oluşturan üyeler `final` tanımlanabilir.
  - Her flyweight nesnesinin paylaşılması gerekmeyebilir. Kardeş sınıf gerekmiyorsa soyut üst sınıf da kaldırılabilir; fabrika doğrudan `ConcreteFlyweight` döndürebilir.
- Flyweight gerçekleme ayrıntıları
  - Hocaya göre fabrika isteğe bağlı değildir, çünkü anahtarla nesne almak her durumda bir havuzla yapılır.
  - Flyweight nesnelerini Singleton yapmaya gerek yoktur; fabrika zaten anahtara kar-

şılık aynı nesneyi döndürür.
  - Erişimi daha sıkı denetlemek için flyweight'in işlem metotları bir arayüzde tanımlanabilir, flyweight sınıfı fabrikanın `private` iç sınıfı yapılabilir; nesnenin kendisi yerine bir klonu ya da handle'ı döndürülebilir. Benzer iç sınıf kullanımı Memento kalıbında yapılacaktır.
- Flyweight'in kullanılabileceği anlar (ekrandan, 00:30:00): aşağıdaki koşulların tümü doğru olmalıdır.
  - Uygulama çok fazla sayıda nesne kullanıyorsa.
  - Nesnelerin çokluğu nedeniyle saklama masrafları çok yüksekse.
  - Nesnelerin durum bilgisi içsel ve dışsal olarak iki bölüme ayrılabiliyorsa.
  - Dışsal durumları ayrılan nesnelerin birçok grubunun yerini paylaşılabilecek az sayıda nesne alabiliyorsa.
  - Uygulama nesne kimliğine bağlı değilse.
- Flyweight örneği: kimya fabrikası (ekrandan, 00:33:30, 00:40:00, 00:41:35)
  - Fabrikada birçok kimyasal madde ve bunların birleşiminden oluşan çok sayıda karışım üretiliyor; bir madde birden fazla karışımda yer alabiliyor.
  - İlk çözüm: `KimyasalMadde` (`gram`, `isim`, `simge`, `atomAgirligi`; `getMol()` = `gram/atomAgirligi`) ve `Karışım` (`isim`, `LinkedList<KimyasalMadde> maddeler`, `maddeEkle()`, `tarifEt()`). Örnek: 100 g barutta 75 g potasyum nitrat, 15 g karbon, 10 g kükürt. Sorun: yalnız gram bilgisi farklı, kalan durum bilgisi aynı çok sayıda karbon nesnesi oluşur.
  - Daha doğru modelleme: değişen gram bilgisi yeni `Bileşen` sınıfına (`gram`, `madde`) alınır. `Bileşen`'in `getMol()` ve `getMaddeAdi()` metotları, işi `KimyasalMadde`'ye ileten delegasyon metotlarıdır; `getMol()` kendi gramını `madde.getMol(gram)` çağrısına parametre verir. Gram dışsal bilgi olarak dikkati çeker.
  - Flyweight ile çözüm: `MaddeFabrikasi` (`static HashMap<String, KimyasalMadde> maddeler`, `static maddeBul(simge)`) eklenir. `Karışım.bileşenEkle()` artık maddeyi değil, anahtar olan simgeyi (`"C"`, `"KNO3"`) alır ve maddeyi fabrikadan ister.
- Proxy (ekrandan, 01:13:20)
  - Amaç: bir nesneye erişimi denetlemek için onun yerine geçecek başka bir nesneyi araya koymak.
  - Örnek: büyük resimlerin çizilmesi ya da indirilmesi zaman alır. Resim yüklenene kadar yerinde küçük bir yer tutucu gösterilir. Hoca, sayfa yüklenirken gri kutular gösteren güncel bir video sitesini örnek olarak açtı: bu yer tutucular insanın sabırsızlığını törpüler.
  - GoF örnek çözümü: `DocumentEditor` → soyut `Graphic` (`Draw()`, `GetExtent()`, `Store()`, `Load()`); `Image` ve `ImageProxy` bundan türer. `ImageProxy`'de `Draw()`: `if (image == 0) { image = LoadImage(fileName); } image->Draw();` `GetExtent()`: resim yüklenmemişse kendi `extent`'ini, yüklenmişse `image->GetExtent()` döndürür.
  - Vekil ile asıl nesne aynı arayüzü desteklemelidir; böylece istemci için şeffaflık korunur.
  - Kalıp yapısı: `Client` → `Subject` (`Request()`); `Proxy` ve `RealSubject` bundan türer, `Proxy.Request()` işi `realSubject->Request()`'e iletir, öncesinde ya da sonrasında ek işler yapabilir.
  - Kullanım: bir nesneye basit bir işaretçiden daha fazlası gerektiğinde: erişim denetimi, gecikmeyi (latency) daha az hissettirme, ödeme kontrolü, yetki kontrolü vb.
- Proxy örneği: izlediğin kadar öde sunucusu (ekrandan, 01:20:00, 01:23:15)
  - Sunucu verilen adrese istenen programı gönderir; göndermeden önce istemcinin ödeme yapıp yapmadığına bakılmalıdır. Sunucu yazılımında bu yetenek yoktur ve kaynak koduna erişilemez. Araya konan vekil bu işi çözer (protection proxy).
  - Var olan: `StreamDestination` (`server : StreamSource`, `address`, `requestProgram()`), `«interface» StreamSource` (`stream(programID, destination) : boolean`), `StreamServer`.
  - Eklenen: `StreamProxy` (`original : StreamSource`, `stream()`, `checkPayment()`) ve `StreamDestination.makePayment(double)`. `stream()` önce `checkPayment()` yapar, ödeme başarılıysa `original.stream()` çağırır.
  - `MainApp`'te gerçek sunucu proxy'nin kurucusuna verilir, istemciye sunucu olarak proxy verilir.
- Chain of Responsibility (davranışsal kalıp) (ekrandan, 01:40:00, 01:44:45, 01:50:00)
  - Amaç: zaten var olan ilişkili bir nesneler zincirinde (tek zincir ya da ağaç hiyerarşisi), bir nesnenin hakkıyla yerine getiremeyeceği bir metot çağrısını, işi daha iyi

yapabilecek başka bir nesneye iletmesi.
  - Örnek: bağlama duyarlı yardım. Kullanıcı bir diyalog kutusundaki bileşen için yardım istediğinde en özelleşmiş yardım gelmeli; o düzeyde yardım yoksa kademeli olarak daha genel yardıma (sekme, diyalog kutusu, uygulama) gidilmeli. GoF şeması: `aPrintButton` → `[!canHelp] HandleHelp()` → `aPrintDialog` → `[!canHelp] HandleHelp()` → `anApplication`.
  - Çözüm: bileşenler mesajın iletimi için ortak bir arayüze sahip olmalıdır; yoksa if-else/switch-case ile üst bileşenin tipi test edilir. Bir nesne yalnız mesajı yanıtlayıp yanıtlayamayacağını bilir.
  - Kalıp yapısı: `Client` → soyut `Handler` (`HandleRequest()`, kendine `successor` ilişkisi); `ConcreteHandler1`, `ConcreteHandler2`. Zincir üyesi tutulacağı için `Handler` arayüz değil soyut sınıf olmalıdır; zincir yönetim metotları da orada kodlanır, yoksa alt sınıflarda tekrarlanan kod oluşur.
  - Uygulanabileceği anlar: bir mesajı yanıtlayabilecek birden fazla nesne varsa ve hangisinin yanıt vereceği önceden bilinemiyorsa; yanıtlayacak nesneler çalışma anında tanımlanabilecekse.
  - Zayıf yönü: zincir doğru yapılandırılmazsa mesajın yanıtlanacağının garantisi yoktur.
  - Gerçekleme: zincir halihazırda yoksa zinciri kuran metotlar `Handler`'da ya da `ConcreteHandler`'larda kodlanır; isteğin birden fazla türü olabiliyorsa ya birden fazla `HandleRequestX` metodu ya da isteği simgeleyen bir kalıtım hiyerarşisi kurulur.
- Chain of Responsibility örneği: reklam yönetim yazılımı (ekrandan, 02:04:50, 02:02:00, 02:10:00)
  - Kampanya 1: spor sitesinin 18-39 yaş arası erkek ziyaretçilerine tıraş bıçağı, aynı yaş aralığındaki kadınlara fitness salonu, diğerlerine spor mağazasındaki indirim reklamı gösterilir.
  - Kampanya 2: haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamı gösterilir; aksi halde kampanya 1 aynen uygulanır.
  - Kampanya 3 (kodlanmadı): ziyaretçinin coğrafi bölgesine ve gelir düzeyine göre lokanta reklamlarından biri gösterilir.
  - Tasarım: `User` (`Cinsiyet` enum, `GelirDuzeyi` enum, yaş, yaş aralığı kontrol metodu); soyut `Reklam` (`banner`, `sonraki`, soyut `reklamUygunMu(User)`, soyut `reklamGoster(User)`, `final reklamKotar(User)`); alt sınıflar `TrasBicak`, `FitnessSalonu`, `IndirimSporMagaza`, `OtoDergi`.
  - Zincirin sonundaki `IndirimSporMagaza` her kullanıcı için `true` döndürür; zincir mutlaka bir noktada sonlanmalıdır.
- Command (davranışsal kalıp) (ekrandan, 02:20:00)
  - Amaç: nesnelere giden istekleri (metot çağrılarını) nesne haline çevirmek. Böylece çağrıları kuyruklamak ya da günlüğünü (log) tutmak kolaylaşır.
  - GUI çerçevelerinde bir düğmeye ya da menü öğesine tıklandığında belirli bir arayüzün metodunu çağıran olay işleme altyapısının arkasındaki tasarım bu kalıptır.
  - GoF örneği: menü öğesi (`MenuItem`) bir dosya açma komutuyla ilişkilendirilir. Soyut `Command` (`Execute()`); `OpenCommand` (`Execute()`, `AskUser()`) `Application`'a (`Add(Document)`) bağlıdır. `Execute()`: `name = AskUser(); doc = new Document(name); application->Add(doc); doc->Open()`.
  - Kalıp yapısı: `Command` (komut tanımlama arayüzü), `ConcreteCommand` (somut komut, `Execute()` içinde işi yapar), `Receiver` (komuttan etkilenen nesne), `Client`, `Invoker` (komutların yürütülmesini isteyen, bir tür istemci).
- Command örneği: akıllı ev kumandası (Head First Design Patterns'ten esinlenme) (ekrandan, 02:26:30, 02:30:00)
  - «`interface`» `AcKapa` (`ac() : boolean`, `kapa() : boolean`); gerçekleyen sınıflar `AydinlatmaDugmesi`, `ElektronikAlet`, `Iklimlendirme` (her biri `isim` tutar).
  - `Kumanda`: düğme listesi, `dugmeEkle()`, `ac(dugmeNo)`, `kapa(dugmeNo)`, `hepsiniKapa()`.
  - `AnaProgram`: kumandayı kurar ve senaryoları (`evBoskenSokakKapisiAcildi()`, `evdeKimseKalmadi()`) tanımlar.

Hocanın Özellikle Vurguladığı Kısımlar

- Flyweight ile Composite'in farkı
  - Kimya örneği ilk bakışta Composite'i çağrıştırabilir. Ama aynı madde birden fazla karışımda ortak kullanılır. Composite'te bir varlık belirli bir anda yalnız bir bütünün

parçası olabilir.
- Nesne kimliği kritik
  - Flyweight'te bütün göstericiler havuzdaki aynı nesneye gider; ayrı nesne kimliği yoktur. Kelime işlemci örneğinde sakıncası yoktur, ama kimliğin önemli olduğu durumlarda (ör. müşteri sayılırken kişilerin mi siparişlerin mi sayıldığı) kalıp kullanılmamalıdır.
- Tasarruf içsel bilginin büyüklüğüne bağlı
  - Kalıbın yarar sağlaması için durum bilgisi içsel ve dışsal olarak ayrılabilmelidir; paylaşılan içsel bilgi ne kadar büyükse tasarruf o kadar artar.
- Neyin değiştiğini bul ve sarmala
  - Kimya örneğinde değişen gram bilgisi ayrı sınıfa alınarak durum bilgisi değişen ve ortak kullanılan iki parçaya bölündü.
- Proxy'de şeffaflık
  - Vekil ve asıl nesne eş biçimde kullanılabilmelidir; istemci farklı nesneleri farklı biçimde kullanmak zorunda kalırsa şeffaflık ve kolaylık kaybolur. Araya boş yere arayüz koymak, yalnız kalıbın getirdiği erişim denetimi gerekiyorsa anlamlıdır.
- if-else zinciri bir alarm sinyalidir
  - Kendinizi if-else-if zinciri yazarken buluyorsanız, büyük olasılıkla bir kalıtım ilişkisi ve çok biçimlilik kullanmanız gerekiyordur.
- Zincir doğru yapılandırılmalı
  - Hoca bunu, bir sorunun memurdan şefe, şube müdürüne, birim müdürüne gidip sonunda hiçbir yerde yanıtlanmaması benzetmesiyle anlattı. Zincir kendi içinde mantıklı sırada kurulmalıdır.
- Ekstra ilişki ekstra bağlaşımdır
  - Command'da alıcıdan komuta doğru bir ilişki gerekmez; gerekmeyen her ilişki bağlaşımı artırır.

Kısa Tekrar Notları

- Facade: alt sistemlerin farklı arayüzlerini tek, üst düzey bir arayüzde toplar.
- Flyweight: çok sayıda basit nesne paylaşımla kullanılır; `FlyweightFactory` anahtara göre havuzdan nesne döndürür.
- İçsel durum (intrinsic, immutable) paylaşılır; dışsal durum (extrinsic, mutable) metot parametresiyle verilir.
- Flyweight koşulları: çok nesne, yüksek saklama masrafı, durumun ayrılabilmesi, az nesnenin çok grubun yerini alabilmesi, nesne kimliğine bağlı olmama (hepsi birden).
- Kimya örneği: `Bileşen` (gram, dışsal) + `KimyasalMadde` (paylaşılan) + `MaddeFabrikasi` (`static` havuz, simge anahtarı).
- Proxy: aynı arayüzü destekleyen vekil, erişimi denetler ve isteği asıl nesneye iletir (ör. resim yer tutucusu, ödeme kontrolü).
- `StreamProxy.stream()`: önce `checkPayment()`, başarılıysa `original.stream()`.
- Chain of Responsibility: istek, yanıtlayabilecek nesne bulunana kadar `successor` zinciri boyunca iletilir; `Handler` soyut sınıf olmalı.
- Reklam örneği: `reklamKotar()` uygunsa gösterir, değilse `sonraki`'ye iletir; zincirin sonu herkese `true` döndürür.
- Command: metot çağrısı nesne olur; `Command.Execute()`, `ConcreteCommand`, `Receiver`, `Invoker`.
- Akıllı ev örneği: `AcKapa` arayüzü, `Kumanda` (düğme listesi), `AnaProgram` (senaryolar).

Detaylı Açıklamalar

- Flyweight'te bellek kazancının mantığı hocanın sayısal anlatımıyla şöyledir: bir milyon harflik bir metinde her konumda yalnız harfin anahtarı tutulursa, UTF-16 için 2 bayt ile yaklaşık 2 milyon bayt, buna havuzdaki harf grafiklerinin kapladığı alan eklenir. Oysa bir milyon harfin her biri 1 KB'lık grafik nesnesi olarak saklansaydı yaklaşık 1 GB gerekirdi.
- Kimya örneğinde `MaddeFabrikasi` (ekrandan, 00:47:45). Hoca, `static` alanı tanımlandığı yerde hem ilklendirip hem de içine ilk değerleri koymak için `static { ... }` bloğunun kullanıldığını, bu gösterimi bu örnekle öğrendiğini söyledi. Bunun desteklenmediği dillerde aynı iş bir `static` başlatma metoduyla yapılabilir; ama o zaman metodu çağırmayı

unutmamak gerekir.

```
package dp.flyweight.solution3;
import java.util.*;
public class MaddeFabrikasi {
    private static HashMap<String, KimyasalMadde> maddeler = new
        HashMap<String, KimyasalMadde>();
    static {
        maddeler.put( "C", new KimyasalMadde("Karbon", "C", 12.0107) );
        maddeler.put( "S", new KimyasalMadde("Kükürt", "S", 32.066) );
        maddeler.put( "KNO3", new KimyasalMadde("Potasyum Nitrat", "KNO3",
            101.103) );
        maddeler.put( "NaNO3", new KimyasalMadde("Sodyum Nitrat", "NaNO3",
            84.9947) );
    }
    public static KimyasalMadde maddeBul( String simge ) {
        return maddeler.get(simge);
    }
}
```

- Fabrikanın `static` olmasının sakıncası yoktur, çünkü bütün karışımların maddeleri aynı fabrikadan gelir. `MainApp` (ekrandan, 00:50:00):

```
package dp.flyweight.solution3;
public class MainApp {
    public static void main(String[] args) {
        Karisim karaBarut = new Karisim();
        karaBarut.setIsim("Kara barut");
        karaBarut.maddeEkle("NaNO3", 75);
        karaBarut.maddeEkle("C", 15);
        karaBarut.maddeEkle("S", 10);
        System.out.println( karaBarut.tarifEt() );
        Karisim temizBarut = new Karisim();
        temizBarut.setIsim("Temiz barut");
        temizBarut.maddeEkle("KNO3", 75);
        temizBarut.maddeEkle("C", 15);
        temizBarut.maddeEkle("S", 10);
        System.out.println( temizBarut.tarifEt() );
    }
}
```

- `tarifEt()`, bir for-each döngüsünde her bileşenin gramını ve madde adını yazarak tarifi döndürür. Kod olarak ikinci ve üçüncü çözüm arasında fazla fark yoktur; fark, her karışımda aynı madde nesnesinin yeniden kullanılmasıyla elde edilen saklama tasarrufundadır.

  Ders sonrası not: Slayttaki kodda iki barutta da 75 + 15 + 10 = 100 g vardır. Derste çalıştırılan sürümde ise "Temiz barut" çıktısı 75 g potasyum nitrat, 20 g karbon ve 25 g kükürt (toplam 120 g) gösterdi (ekrandan, 02:00:00); hoca toplamın 100 etmediğini fark edip doğru oranları sonra kontrol edeceğini söyledi.

  Ders sonrası not: Hoca (00:27:04) Java'nın her nesneye bir "universal unique identifier" koyduğunu söyledi. Java nesnelere UUID atamaz; nesne kimliği, referansların `==` ile karşılaştırılmasıyla belirlenir. Hocanın asıl vurgusu (farklı nesnelerin bellekte farklı yerlerde durup ayrı kimliğe sahip olması) doğrudur.

- Proxy örneğinde `MainApp`, `StreamServer` nesnesini oluşturur, bunu `StreamProxy`'nin kurucusuna verir ve `StreamDestination`'ı bir adresle (`"12:aa:34:ff:54"`, ekrandan, 01:30:00) kurup sunucu olarak proxy'yi atar. İstemci program istediğinde aslında proxy'ye başvurur; proxy ödeme yapılmışsa gerçek sunucuya başvurur. `checkPayment()` ödeme yapılmışsa `true`, aksi halde `false` döndürecek biçimde iskelet olarak bırakıldı.
- Reklam örneğinde soyut `Reklam` sınıfı (ekrandan, 02:02:00):

```
public abstract class Reklam {
    private Banner banner;
    private Reklam sonraki;
    public Reklam(Banner banner, Reklam sonraki) {
        this.banner = banner; this.sonraki = sonraki;
    }
    public abstract boolean reklamUygunMu ( User u );
    public abstract void reklamGoster( User u );
    public final void reklamKotar( User u ) {
        if( reklamUygunMu(u) ) {
            System.out.println( this.getClass().getName()
                + "->" + u );
            reklamGoster(u);
        }
        else if( sonraki != null ) {
            System.out.println( this.getClass().getName()
                + "->" + sonraki.getClass().getName());
            sonraki.reklamKotar(u);
        }
    }
}
```

- `reklamKotar()` `final` olduğu için alt sınıflar onu kalıtımla aynen alır; yeni bir reklam yazan uzman kullanıcı yalnız `reklamUygunMu()` ve `reklamGoster()`'i doldurur. Reklam uygun değilse ve sonraki yoksa hiçbir şey yapılmaz.
- `Cinsiyet` enum'u (ekrandan, 02:00:00) nesnel (non-primitive) bir enum örneğidir: `KADIN("Kadın"), ERKEK("Erkek")`, `private String tanim`, `private` kurucu ve `toString()` override'ı. `GelirDuzeyi` ise yalın `A, B, C` enum'udur.
- Kampanya 1 (ekrandan, 02:10:00): zincir `TrasBicak → FitnessSalonu → IndirimSporMagaza → null` biçiminde, dizinin sonundan başa doğru kurulur ve `reklamlar[0].reklamKotar(users[i])` ile her kullanıcı için çalıştırılır. Çıktı: Erkek, C, 23 → `TrasBicak`; Erkek, A, 53 → `TrasBicak → FitnessSalonu → IndirimSporMagaza`; Kadın, B, 33 → `TrasBicak → FitnessSalonu`.
- Kampanya 2'de zincirin başına `OtoDergi` eklenir. Hoca 28 yaşında A gelir grubundan bir erkek kullanıcı ekleyip çalıştırdı: `OtoDergi` C grubundaki erkeğe kendini göstermedi, A grubundaki 28 yaşındaki erkeğe gösterdi. Zincir bir bağlı listede tutulsaydı kampanya 2'nin kampanya 1'i içermesi daha az tekrarlı kodla yazılabilirdi; hoca bunu alıştırma olarak bıraktı.
- Akıllı ev örneğinde `AnaProgram` (ekrandan, 02:30:00):

```
public class AnaProgram {
    Kumanda kumanda;
    public AnaProgram( ) {
        kumanda = new Kumanda();
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Antre" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Salon" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Mutfak" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Oda" ) );
        kumanda.dugmeEkle( new ElektronikAlet( "Vestel TV" ) );
        kumanda.dugmeEkle( new Iklimlendirme( "Kombi" ) );
    }
    public void evBoskenSokakKapisiAcildi( ) {
        kumanda.ac(0); kumanda.ac(5);
    }
    public void evdeKimseKalmadi( ) { kumanda.hepsiniKapa( ); }
    public static void main(String[] args) {
        AnaProgram prg = new AnaProgram();
        prg.evBoskenSokakKapisiAcildi();
        System.out.println("Evden çıkılıyor");
        prg.evdeKimseKalmadi();
    }
```

```
}
```

- `Kumanda.ac(dugmeNo)` önce o numarada bir düğme olup olmadığına bakar, yoksa `false` döndürür, varsa düğmenin `ac()` metodunu çağırıp sonucunu döndürür; `kapa()` benzerdir, `hepsiniKapa()` listedeki bütün düğmelerin `kapa()` metodunu çağırır.

  Ders sonrası not: Hoca `evBoskenSokakKapisiAcildi()` senaryosunu "girişin ışığı ve televizyon açılsın" diye anlattı. Listede 0 numara "Antre", 4 numara "Vestel TV", 5 numara "Kombi"dir; bu yüzden `kumanda.ac(5)` büyük olasılıkla televizyonu değil kombiyi açar.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.